

UNIT-3

* Syntax directed Translation *

* Syntax directed translation (SDT) means attaching meaning to grammar rules during parsing.

* It combines Syntax (structure) & Semantics (meaning) in the compiler.

* SDT is used to perform task evaluating expression or generating code or building syntax tree while parsing the input.

$$[SDT = Grammar + Semantic Rule]$$

Syntax directed Derivations

Synthesized Attributes

Inherited Attributes

Syntax directed translation scheme

$A \rightarrow BCD$

$\{A.i = B.i, C.i, D.i\}$

$C.i = A.i$

$C.i = B.i$

$C.i = D.i$

First at N.T or Value then children are calculated & then

Q1 Construct the parser for exp

$$E \rightarrow E' + T$$

$$E \rightarrow T$$

$$T \rightarrow T' * F$$

$$T \rightarrow F$$

$$F \rightarrow \text{num}$$

$$(3 * 5 * 4) = 22$$

Solⁿ

① SDT

$$E \rightarrow E' + T \quad \{ E.\text{value} = E'.\text{value} + T.\text{value} \}$$

$$E \rightarrow T \quad \{ E.\text{value} = T.\text{value} \}$$

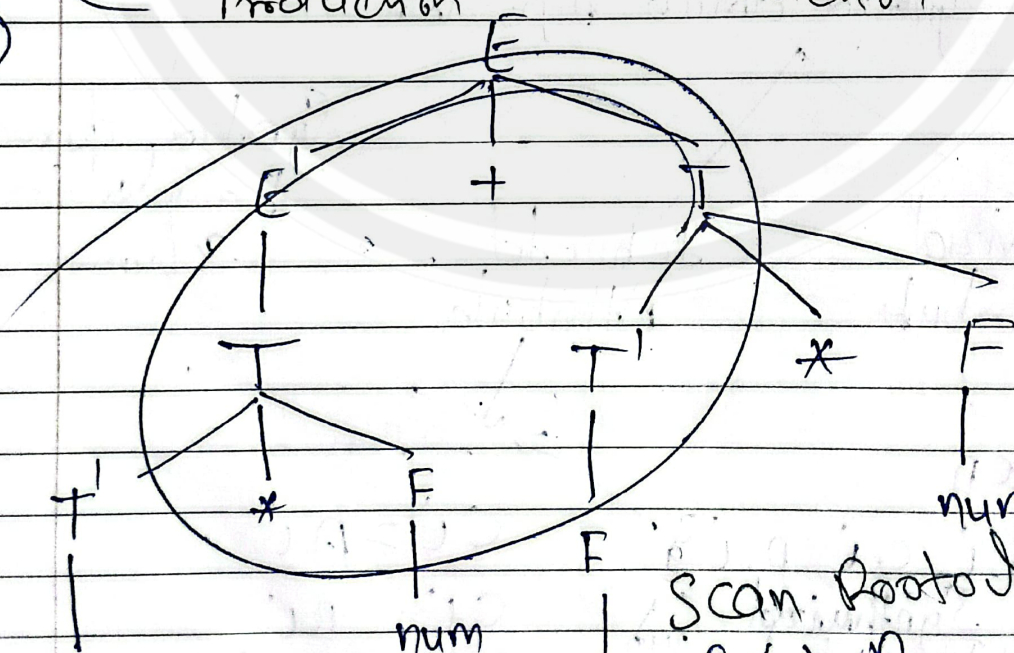
$$T \rightarrow T' * F \quad \{ T.\text{value} = T'.\text{value} * F.\text{value} \}$$

$$T \rightarrow F \quad \{ T.\text{value} = F.\text{value} \}$$

$$F \rightarrow \text{num} \quad \{ F.\text{value} = \text{num}.\text{value} \}$$

Production action

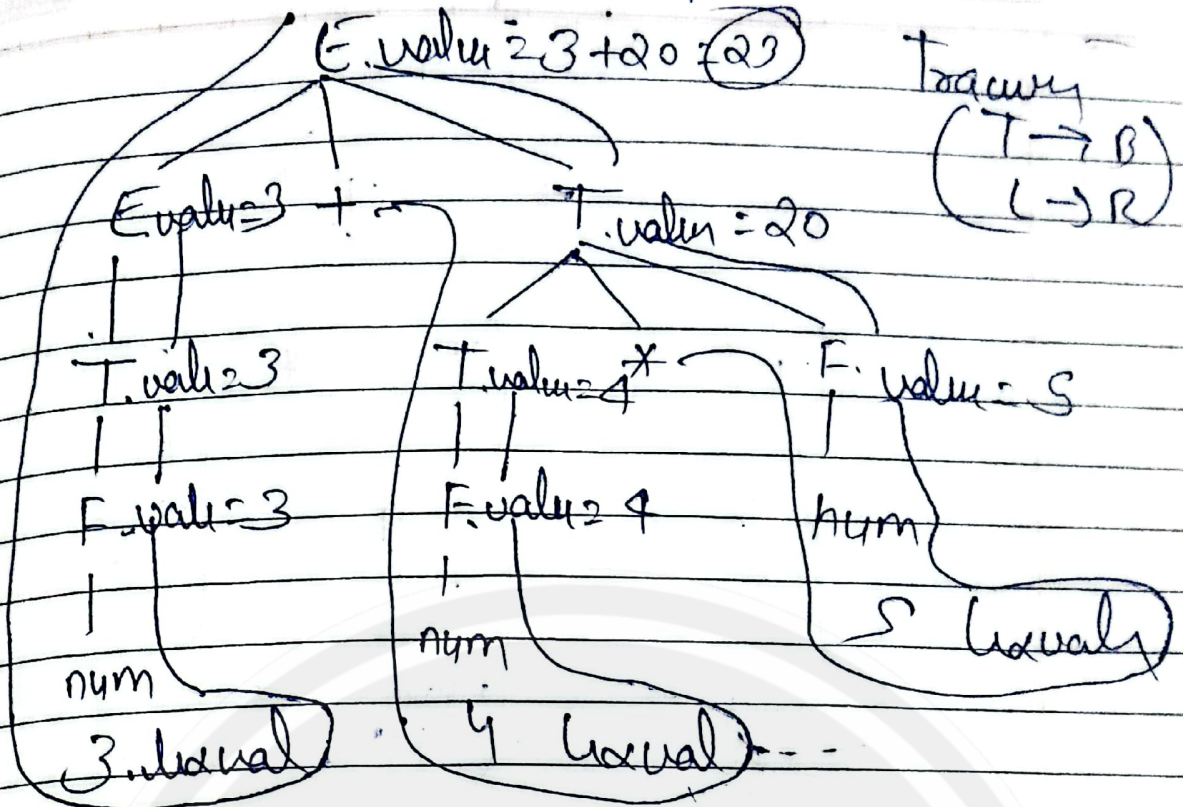
②



Scan. Root Node
num & L & R

Draw tree with EBNF 3rd part
Calc first

(2)



Ex. 2. Infix to Postfix

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow \text{num}$

$3 + 4 * 5$

$3 4 5 * +$

Sol 1 SDT

$E \rightarrow E + T$ { print (+) }

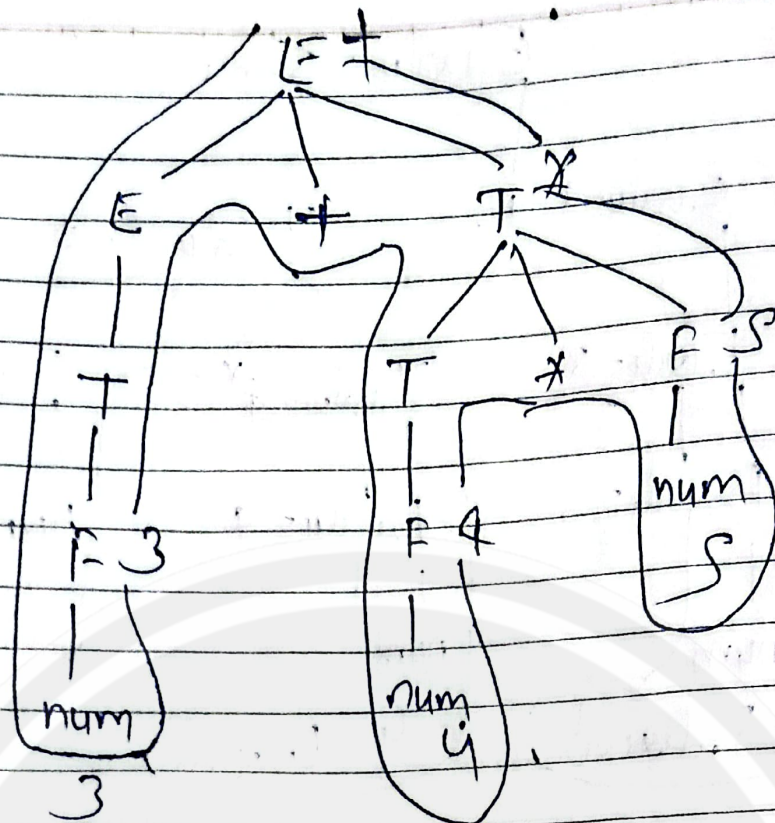
$E \rightarrow T$ { print () }

$T \rightarrow T * F$ { print (*) }

$T \rightarrow F$ { }

$F \rightarrow \text{num}$ { print (value) }

Q



349x+ ✓

Ques

Construct SDT for Arithmetic calculator

$S \rightarrow E\$$
 $E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow I$
 $I \rightarrow I \text{ digit}$
 $I \rightarrow \text{digit}$

23 * 5 + 4

= 119

solⁿ

①

Construct SDT -

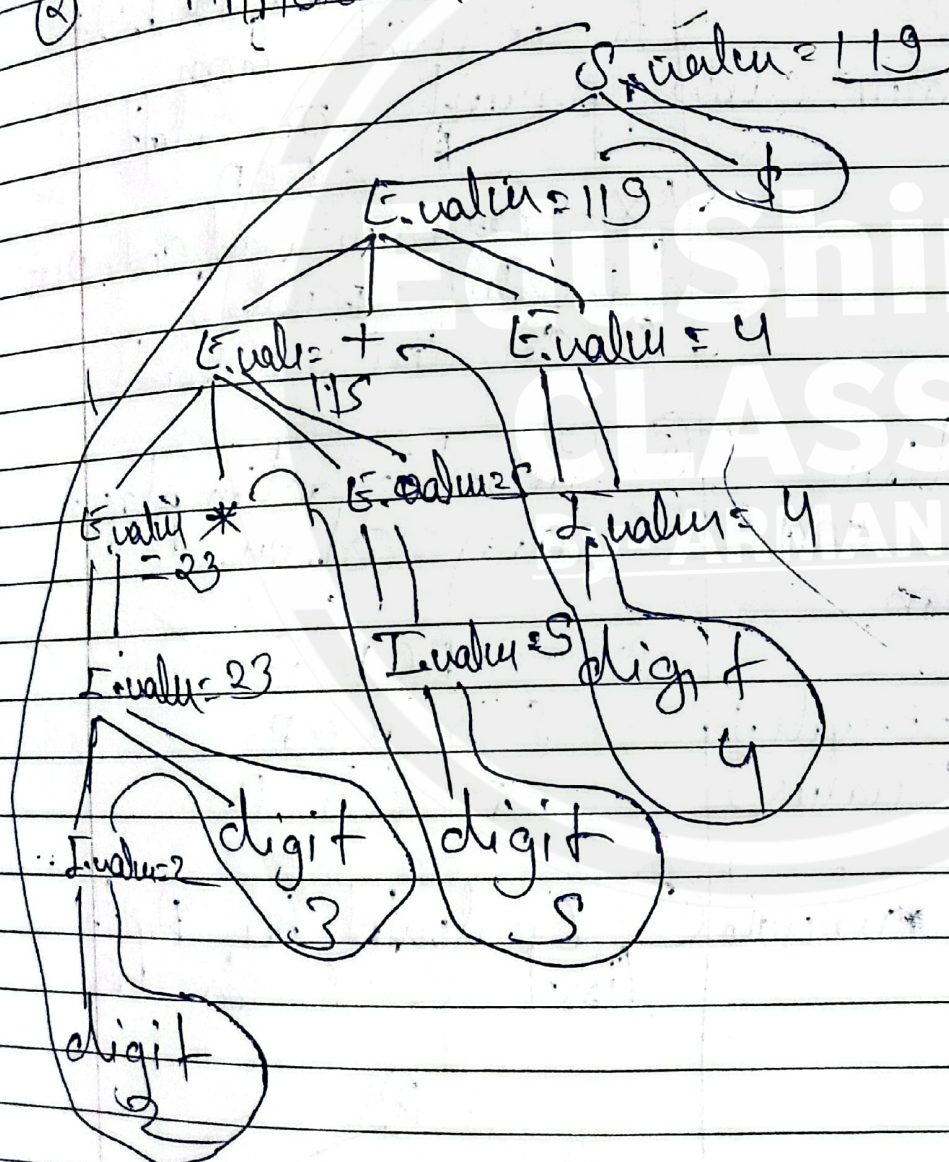
$S \rightarrow E\$ \{ S.value = \text{print } E.value \}$
 $E \rightarrow E + E \{ E.value = G.value + E.value \}$
 $E \rightarrow E * E \{ E.value = E.value * E.value \}$
 $E \rightarrow I \{ E.value = I.value \}$

~~$I \rightarrow I \{ I.value = \dots \}$~~

$I \rightarrow I \text{ digit} \{ I.value = 10 * I.value + \text{digit} \}$

$I \rightarrow \text{digit} \{ I.value = \text{digit}.lexval \}$

② Annotated Parse tree -



Types of SDF

① S-Attributed SDF

② L-Attributed SDF

S-Attributed SDF L-Attributed SDF

① Can only Synthesize
Attribut.

Ex.

$\{A.val = B.val + C.val\}$

① can both Synthesize
and inherit, each
inherited in which
to inherit from
parent or left.

Ex - $\{A.val = B.val + C.val\}$
 $\{B.val = A.val\}$
 $\{C.val = B.val\}$

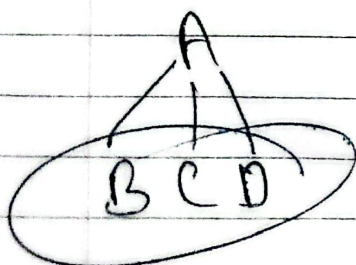
② Semantic action
placed at right end
of production.

$E \rightarrow E + T \{E.val = E.val + T.val\}$

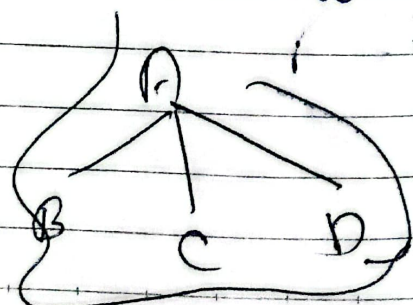
② Semantic action placed
any where.

$A \rightarrow BCD \{ \}$
 $\{ \} BCD$
 $B \{ \} D$

③ Attributes are evaluated
BVP manner



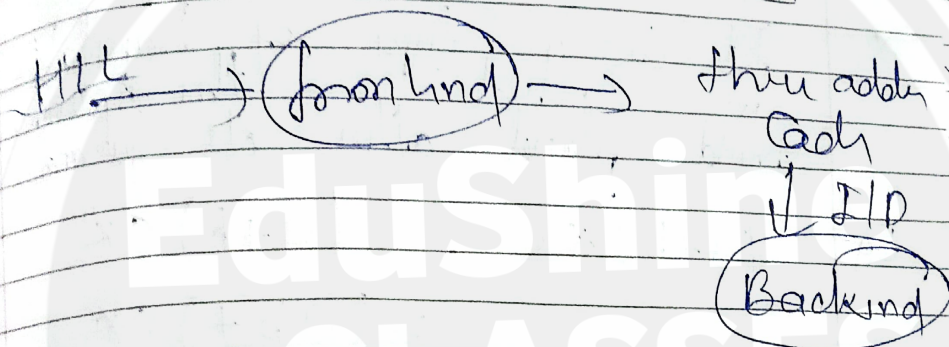
③ Evaluated by DFT
first R L to R



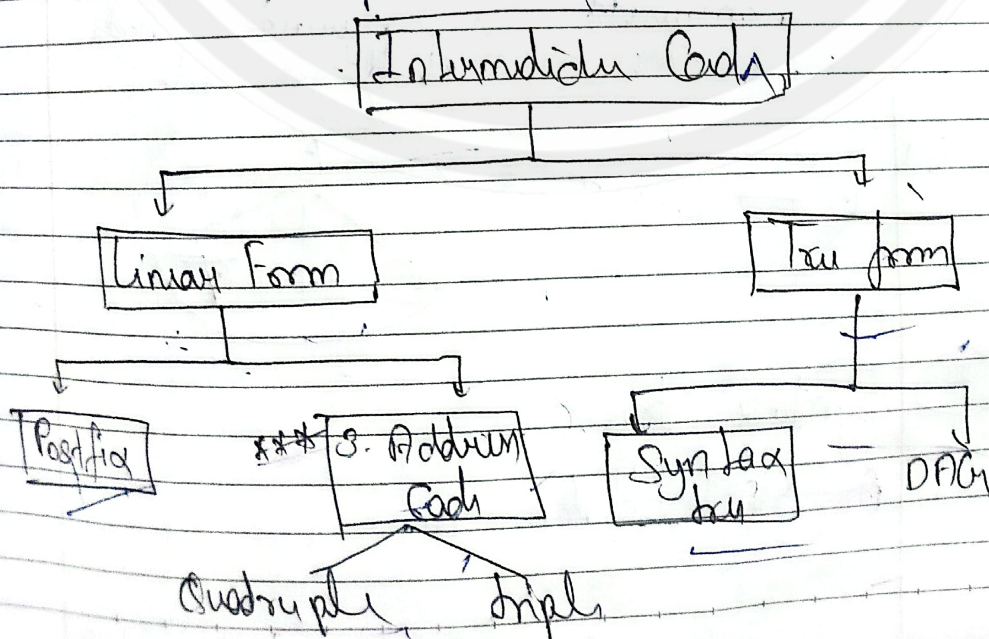
Ex $A \rightarrow BC \quad \{ B.S - A.S \}$

(~~for~~ ~~the~~ ~~attributed~~ ~~SDT~~ ✓)

Intermediary Code Generation



* Intermediary Code generation is the fourth phase of compiler which takes parse tree as input from semantic phase & generates intermediary code as output.



① Postfix Notation

* Postfix Notations are also known as reverse polish notation.

* It is a type of translation in which the operator should placed after its operand.

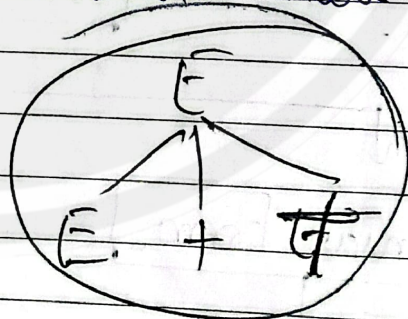
Eg- $(a+b) * c = (a+b) c *$
 $= (ab+) c *$

② Parse Tree

* A Parse Tree is an ordered tree in which left hand side of a production of grammar represent a parent node & R.H.S children node.

Eg

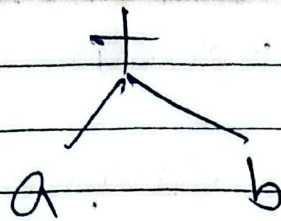
$$E \rightarrow E + T$$



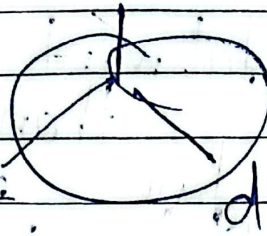
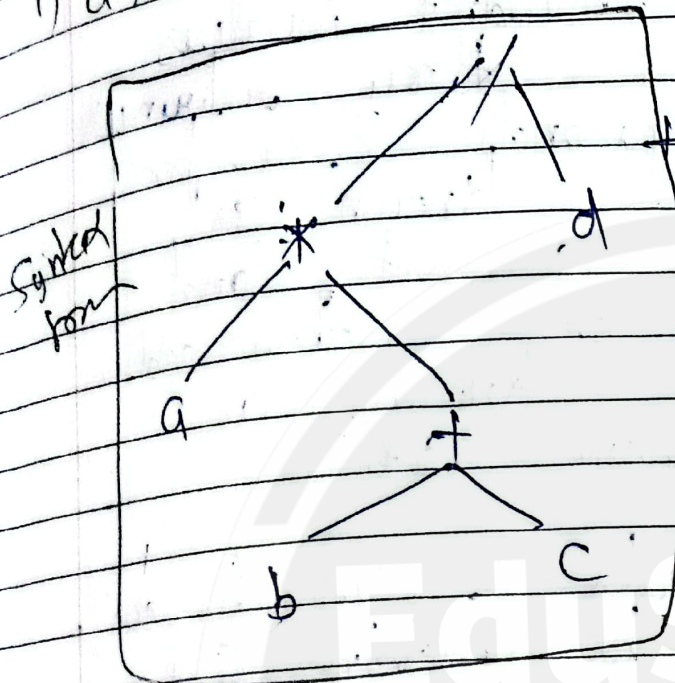
③ Syntax tree

A Syntax tree is

eg - $a + b$



i) $a * (b + c) / d$



first calculate
↓
mean
left side

Now we can find
infix, prefix, postfix

Infix - L R R
prefix - Root Left Right
postfix - Left Right Root

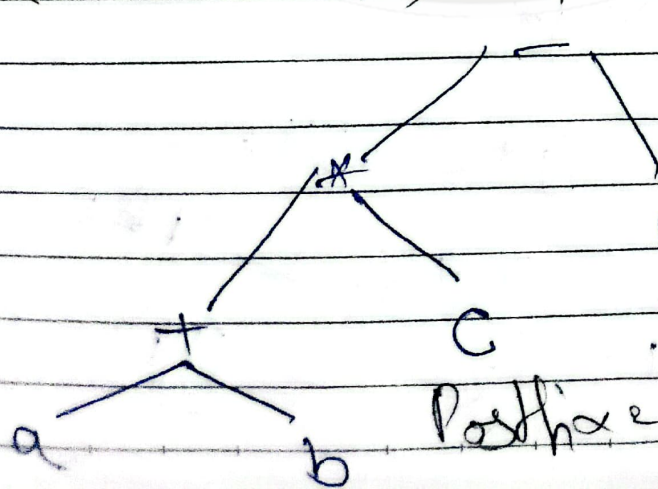
Infix - $a * b + c / d$ ✓

Prefix - ~~$* a b +$~~ $/ * a + b c d$ ✓

Postfix - $a b c + * d /$ ✓

ii) $((a + b) * c) - d$

L R R R



Can be taken
as intermediate
code

Postfix - $a b + c * d -$

④ Thru Address Code -

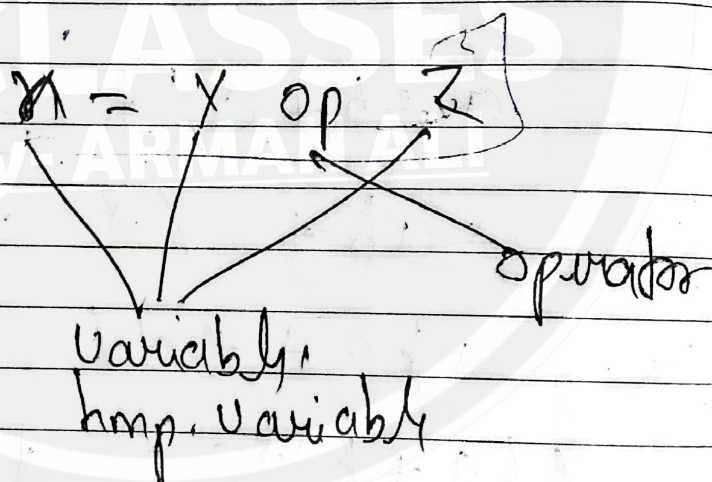
* Thru address Code is a type of jump of instruction Code.

* A statement shouldn't involve more than three operands (two for operands & one for result) is known as thru address statement and a sequence of thru address statements is known as thru address Code.

* Easy to generate & can be easily converted to machine Code.

Format

total 3 address
3 variable
& some are
i/o



Eg - $a = r + y * z$

$t_1 = y * z$ TAC

$t_2 = r + t_1$ TAC

$a = t_2$ TAC

* Types of three address statements -

① Logical or Binary op^r

$$X = Y \text{ op } Z$$

$$X = Y + Z$$

② Unary operator

$$X = \text{op } Y$$

③ Copy statement

$$X = Y$$

④ Procedure call

param X
param Y

⑤ Index Assignment

$$X = Y[i]$$

$$X[i] = Y$$

⑥ Conditional jump if $X \text{ rel.op } Y$ goto label

$$< > =$$

⑦ Address & pointer

$$X = \&Y$$

$$X = *X$$

Implementation

① Quadruple - $\text{arg1}, \text{arg2}, \text{op}, \text{result}$

Ex - $a = b * -c + b * -c$

→ First Convert in three address code

$$t_1 = -c$$

$$t_2 = b * t_1$$

$$t_3 = -c$$

$$t_4 = b * t_3$$

$$t_5 = t_2 + t_4$$

$$a = t_5$$

Quadruple table -

	Op	arg1	arg2	Result
0	unary (-)	c		t_1
1	*	b	t_1	t_2
2	unary (-)	c		t_3
3	*	b	t_3	t_4
4	+	t_2	t_4	t_5
5	assign	t_5		a

Q. Triple -

Op, arg1, arg2

Triple table -

	Op	arg1	arg2
0	unary(-)	c	
1	*	b	(0)
2	unary(-)	c	
3	*	b	(2)
4	+	t2	(3)
5	arith	as	(4)

Q. With the Quadruple, triple for following -

$$(n+y) * (y+z) + (n+y+z)$$

sol^y

Then address Calc -

$$\begin{aligned} t_1 &= n+y \\ t_2 &= y+z \\ t_3 &= n+y+z \end{aligned}$$

~~$$t_4 = t_1 + t_2 + t_3$$~~

$$t_1 = n+y$$

$$t_2 = y+z$$

$$t_3 = t_1 * t_2$$

~~$$t_4 = n+y+z$$~~
$$t_4 = t_1 + t_2$$

$$t_5 = t_3 + t_4$$

Quadruple table

	op	arg1	arg2	Result
0	+	x	y	t ₁
1	+	y	z	t ₂
2	*	t ₁	t ₂	t ₃
3	+	t ₁	z	t ₄
4	+	t ₃	t ₄	t ₅

Now

Triple table ~

	op	arg1	arg2
0	+	x	y
1	+	y	z
2	*	(0)	(1)
3	+	(0)	z
4	+	(2)	(3)

Q

Give the then address code for following program -

```

int i
i = 1
while i < 10 do
  if n > y then
    i = n + y
  else
    i = n - y
  
```

declaration of variable then address code

Sl Then address code -

```

1 i = 1
2 if i < 10 go to (4)
3 go to (10)
4 if n > y go to (6)
5 go to (8)
6 i = n + y
7 go to (2)
8 i = n - y
9 go to (2)
10 Stop
  
```

Q

```

c = 0
do {
  if a < b then
    n++
  else n--
  c++
} while (c < 5);
  
```

```

1 c = 0
2 if a < b go to (4)
3 go to (7)
4 n = n + 1
5 c = c + 1
6 go to (3)
7 n = n - 1
8 go to (5)
  
```

* More About Translation

- ① Array References in Arithmetic
- ② Procedure Call
- ③ Can Statement
- ④ ~~Procedure Call~~

* Array Reference in Arithmetic

- i) 2D Array
- ii) 2D Array

Ex. int A[5] = {1, 2, 3, 4, 5};

Representation -

Actual Address	1100	1104	1108	1112	1116
Element	1	2	3	4	5
Index	0	1	2	3	4

↑
LB

✓ Address of $A[i] = B + w * (i - LB)$

eg - $B = 1100, LB = 0, w = 4$

$$A[4] = 1100 + 4 * 4$$

4 index = 1116

```

Ex
int i
int a[10]
i = 1
while (i < 10)
{
    a[i] = 0;
    i = i + 1;
}

1 i = 1
2 if i < 10 goto (4)
3
4 goto (8)
5 f1 = 4 * i
6 a[i] = 0
7 i = i + 1
8 goto (2)
9 stop
10
    
```

* 2D Array -

a[3][4]

→ Cal

	LP	1	2	3
row	0	8	6	3
1	4	5	9	1
2	6	3	2	4

Row major

8	6	3	2	4	5	9	1	6	3	2	4
row 0				row 1				row 2			

Column major

8	4	6	6	5	3	3	9	2	2	1	4
Col 1			Col 2			Col 3			Col 4		

m-row
N-cal

Row major formula

$$A[i][j] = B + w * [N * (i - L_r) + (j - L_c)]$$

Ex -

$$A[2][2] = 10000 + 4 * [9 * (1 - 0) + (2 - 0)]$$

$$= 10024$$

Column major formula -

$$A[i][j] = B + w * (i - L_r) + m * (j - L_c)$$

Ex -

int i
int a[10][10]

i = 0

while (i < 10)

{ a[i][i] = 1
i++ }

$$B + 4 * [10 * (i - 0) + (i - 0)]$$

$$B + 4 * (10i + i)$$

$$B + 44i$$

Sol

```

1 i = 0
2 if i < 10 goto (4)
3 goto (8)
4 A[i][i] = 1
5 i = i + 1
6 goto (2)
7 stop

```

Can statement -

switch (ch)

True

Ex

{
 can 1 : $c = a + b$;
 break;

 can 2 : $c = a - b$;
 break;

এই গঠন পড়ে if
এবং else if
এবং break
নিয়ে করা
হয়।

~~1 if ch == 1 goto (5)
2 goto ()
3 if ch == 2 goto ()
4 goto ()
5 $c = a + b$
6 stop~~

1 if ch == 1 goto (3)
2 if ch == 2 goto (5)
3 $c = a + b$
4 goto (7)
5 $c = a - b$
6 goto (7)
7 stop
8

Procedural Call :-

$P(A_1, A_2, A_3 \dots A_n)$

param A_1
param A_2
param A_3
⋮
param A_n

→ Call P ; $n \rightarrow$ No. of actual parameter
 ↓
 Name of procedure

Ex. void main ()
{

int x, y;
--

swap (&x, &y)
--

}

void swap (int *a, int *b)

{
int i;
i = *b
*b = *a
*a = i;
}

1 Call main

2 param &x

3 param &y

4 Call swap;

5 i = *b

6 *b = *a

7 *a = i

8 Stop.

9
10